

beginning

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

data loading

```
df_encoded = pd.read_excel("/content/drive/MyDrive/capstone/datasets/5_encoded.xlsx")
```

```
df_encoded.head()
```

	age	latitude	longitude	feels_like	temp_min	temp_max	air_pressure	air_humidity	wind_speed	wind_deg	...	gender_encoded_male	gender_encoded_third gender	workplace_encode
0	NaN	23.960600	91.119089	302.86	299.15	299.15	1008.0	83.0	2.10	320.0	...	1.0	0.0	
1	20.0	24.899220	91.868527	299.17	295.57	295.57	1012.0	96.0	1.39	116.0	...	1.0	0.0	
2	14.0	23.832984	89.966688	304.33	301.15	301.15	1004.0	78.0	3.60	190.0	...	0.0	0.0	
3	NaN	24.850066	89.372843	303.67	299.34	299.34	1003.0	91.0	2.64	186.0	...	0.0	0.0	
4	18.0	23.764386	90.389014	304.49	300.15	300.15	1004.0	83.0	1.99	154.0	...	0.0	0.0	

5 rows × 37 columns

```
df_encoded.shape
```

```
(1617, 37)
```

```
def missing_data_report(df):
    # Calculate the percentage of missing data for each column
    missing_percentage = df.isna().mean() * 100
```

```
# Get the data types of each column
data_types = df.dtypes

# Create a DataFrame for the missing data summary including data types
missing_data_summary = pd.DataFrame({
    'Column': missing_percentage.index,
    'Missing Percentage': missing_percentage.values,
    'Data Type': data_types.values
})

# Define the four groups based on missing percentage
group_1 = missing_data_summary[missing_data_summary['Missing Percentage'] > 70][['Column', 'Missing Percentage', 'Data Type']]
group_2 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 40) & (missing_data_summary['Missing Percentage'] <= 70)][['Column', 'Missing Percentage', 'Data Type']]
group_3 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 20) & (missing_data_summary['Missing Percentage'] <= 40)][['Column', 'Missing Percentage', 'Data Type']]
group_4 = missing_data_summary[missing_data_summary['Missing Percentage'] <= 20][['Column', 'Missing Percentage', 'Data Type']]

# Sort each group by missing percentage in descending order (most missing to least missing)
group_1 = group_1.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_2 = group_2.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_3 = group_3.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_4 = group_4.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)

# Rename columns for clarity
group_1.columns = ['Column', 'Missing %', 'Data Type']
group_2.columns = ['Column', 'Missing %', 'Data Type']
group_3.columns = ['Column', 'Missing %', 'Data Type']
group_4.columns = ['Column', 'Missing %', 'Data Type']

# Display each group one by one, from most missing to least missing
print("Group 1: Columns with >70% missing data")
print(group_1.to_string(index=False))
print("\nGroup 2: Columns with >40% to ≤70% missing data")
print(group_2.to_string(index=False))
print("\nGroup 3: Columns with >20% to ≤40% missing data")
print(group_3.to_string(index=False))
print("\nGroup 4: Columns with ≤20% missing data")
print(group_4.to_string(index=False))
```

```
missing_data_report(df_encoded)
```

```
Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []
```

```
Group 2: Columns with >40% to ≤70% missing data
      Column  Missing %  Data Type
      feels_like  53.061224  float64
      air_humidity  53.061224  float64
      wind_deg  53.061224  float64
      clouds_sky  53.061224  float64
      weather_main_encoded  52.999382  float64
      weather_description_encoded  52.999382  float64
      financial_condition_encoded  45.516388  float64
```

```
Group 3: Columns with >20% to ≤40% missing data
```

Column	Missing %	Data Type
workplace_encoded_semi-urban	37.909709	float64
workplace_encoded_urban	37.909709	float64
workplace_encoded_rural	37.909709	float64
air_pressure	30.426716	float64
wind_speed	30.179344	float64
temp_min	29.870130	float64
temp_max	29.560915	float64
marital_status_encoded	23.933210	float64

Group 4: Columns with $\leq 20\%$ missing data

Column	Missing %	Data Type
profession_group_encoded	18.923933	float64
reason_encoded	16.821274	float64
age	12.739641	float64
time_encoded	12.059369	float64
age_group_encoded	7.421150	float64
latitude	6.493506	float64
longitude	6.493506	float64
method_encoded	2.906617	float64
suicide_month	2.411874	float64
suicide_day	2.411874	float64
suicide_weekday_encoded	2.411874	float64
suicide_week	2.411874	float64
suicide_year	2.411874	float64
religion_encoded_buddhist	2.411874	float64
religion_encoded_muslim	2.411874	float64
religion_encoded_indigenous	2.411874	float64
religion_encoded_hindu	2.411874	float64
religion_encoded_christian	2.411874	float64
hometown_encoded	1.669759	float64
gender_encoded_third gender	0.432900	float64
gender_encoded_female	0.432900	float64
gender_encoded_male	0.432900	float64

df_encoded.columns

```
Index(['age', 'latitude', 'longitude', 'feels_like', 'temp_min', 'temp_max',
      'air_pressure', 'air_humidity', 'wind_speed', 'wind_deg', 'clouds_sky',
      'time_encoded', 'suicide_month', 'suicide_day', 'suicide_year',
      'suicide_week', 'age_group_encoded', 'financial_condition_encoded',
      'weather_main_encoded', 'suicide_weekday_encoded',
      'profession_group_encoded', 'hometown_encoded', 'reason_encoded',
      'method_encoded', 'weather_description_encoded',
      'marital_status_encoded', 'gender_encoded_female',
      'gender_encoded_male', 'gender_encoded_third gender',
      'workplace_encoded_rural', 'workplace_encoded_semi-urban',
      'workplace_encoded_urban', 'religion_encoded_buddhist',
      'religion_encoded_christian', 'religion_encoded_hindu',
      'religion_encoded_indigenous', 'religion_encoded_muslim'],
      dtype='object')
```

```
import pandas as pd
```

```
def compute_sorted_correlations(df, threshold=0.0):
    corr_matrix = df.corr()

    result = []
```

```

for col in corr_matrix.columns:
    # Drop self-correlation
    related = corr_matrix[col].drop(labels=[col])

    # Keep only correlations above threshold (excluding exact 0)
    related_filtered = related[related.abs() > threshold]

    # Sort by absolute correlation descending
    related_sorted = related_filtered.reindex(related_filtered.abs().sort_values(ascending=False).index)

    # Format results
    related_list = [
        {'related_column': other_col, 'related_value': round(related_value, 4)}
        for other_col, related_value in related_sorted.items()
    ]

    result.append({
        'column_name': col,
        'related_columns': related_list
    })

return result

```

```
correlation_dict = compute_sorted_correlations(df_encoded)
```

```

from pprint import pprint
display(correlation_dict[:2]) # preview first 2 entries

```

[Show hidden output](#)

```
df_encoded.columns
```

```

Index(['age', 'latitude', 'longitude', 'feels_like', 'temp_min', 'temp_max',
      'air_pressure', 'air_humidity', 'wind_speed', 'wind_deg', 'clouds_sky',
      'time_encoded', 'suicide_day', 'suicide_month', 'suicide_year',
      'suicide_week', 'age_group_encoded', 'financial_condition_encoded',
      'weather_main_encoded', 'suicide_weekday_encoded',
      'profession_group_encoded', 'hometown_encoded', 'reason_encoded',
      'method_encoded', 'weather_description_encoded',
      'marital_status_encoded', 'gender_encoded_female',
      'gender_encoded_male', 'gender_encoded_third_gender',
      'workplace_encoded_rural', 'workplace_encoded_semi-urban',
      'workplace_encoded_urban', 'religion_encoded_buddhist',
      'religion_encoded_christian', 'religion_encoded_hindu',
      'religion_encoded_indigenous', 'religion_encoded_muslim'],
      dtype='object')

```

```

from tqdm import tqdm
from sklearn.impute import KNNImputer
from copy import deepcopy

# Start with a deep copy of the dataset so we don't affect the original
df_imputed_knn_context = deepcopy(df_encoded)
df_imputed_knn_no_context = deepcopy(df_encoded)

# === Define constraints ===

```

```

constraints = {
    'age': (0, 100),
    'latitude': (20.57, 26.63),      # Bangladesh latitude range
    'longitude': (88.0167, 92.6833), # Bangladesh longitude range
    'feels_like': (2.6, 300.65),     # Temperature range in Kelvin (lowest and highest)
    'temp_min': (2.6, 300.65),      # Same temp range
    'temp_max': (2.6, 300.65),
    'air_pressure': (1000, 1050),    # ~1000 hPa to a bit above 1020 hPa
    'air_humidity': (70, 100),       # Percent
    'wind_speed': (3, 20),           # ~3 to 16+ km/h, added small buffer
    'wind_deg': (0, 360),           # Wind direction in degrees, full circle
    'clouds_sky': (0, 100),         # Cloudiness in percent
    'time_encoded': (0, 23),        # Assuming time encoded as hour in 24h format
    'suicide_day': (1, 31),         # Day of month
    'suicide_month': (1, 12),       # Month number
    'suicide_year': (2017, 2025),   # Year range you gave
    'suicide_week': (1, 52),        # Week number (approximate)
    'suicide_weekday_encoded': (0, 6),
    'age_group_encoded': (0, 6),
    'financial_condition_encoded': (0, 4),
    'weather_main_encoded': (0, 7),
}

```

```

def impute_with_knn_context(df, context_info, constraints={}, n_neighbors=5):
    for entry in tqdm(context_info, desc="KNN Context Imputation"):
        column_name = entry['column_name']
        predictors = [r['related_column'] for r in entry['related_columns'] if r['related_column'] in df.columns]

        missing_mask = df[column_name].isna()
        imputed_count = 0
        skipped_count = 0
        errors = []

        if not predictors or missing_mask.sum() == 0:
            continue

        df_subset = df[predictors + [column_name]]

        try:
            imputer = KNNImputer(n_neighbors=n_neighbors)
            imputed_array = imputer.fit_transform(df_subset)
            imputed_df = pd.DataFrame(imputed_array, columns=df_subset.columns, index=df_subset.index)

            mask = df[column_name].isna()
            df.loc[mask, column_name] = imputed_df.loc[mask, column_name]
            imputed_count = mask.sum() - df[column_name].isna().sum()

            if column_name in constraints:
                lower, upper = constraints[column_name]
                df[column_name] = df[column_name].clip(lower=lower, upper=upper)

        except Exception as e:
            errors.append(str(e))

        print(f"\n'{column_name}' Imputation Summary (Context):")
        print(f" - Imputed values: {imputed_count}")

```

```

print(f" - Skipped (no predictors): {skipped_count}")
print(f" - Errors: {len(errors)}")
if errors:
    for err in errors[:5]:
        print(f" - Error: {err}")

```

```

return df

```

```

# def impute_with_knn_context(df, context_list, constraints=None, n_neighbors=5):
#     for entry in tqdm(context_list, desc="KNN Context Imputation"):
#         column_name = entry['column_name']
#         related_columns = [rel['related_column'] for rel in entry['related_columns']]

#         if column_name not in df.columns or df[column_name].isnull().sum() == 0:
#             continue # Skip if no missing values to impute

#         context_cols = [col for col in related_columns if col in df.columns]
#         if not context_cols:
#             print(f"Skipping '{column_name}': no valid context columns found.")
#             continue

#         columns_for_imputation = context_cols + [column_name]
#         df_subset = df[columns_for_imputation]

#         try:
#             # Fit KNNImputer on full subset (including missing in context columns)
#             imputer = KNNImputer(n_neighbors=n_neighbors)
#             imputed_array = imputer.fit_transform(df_subset)

#             # Extract the imputed target column (last column)
#             imputed_target = imputed_array[:, -1]
#             imputed_series = pd.Series(imputed_target, index=df_subset.index)

#             # Only update missing values in original dataframe at matching indices
#             mask = df[column_name].isnull()
#             df.loc[mask, column_name] = imputed_series[mask]

#             # Apply constraints if provided
#             if constraints and column_name in constraints:
#                 min_val, max_val = constraints[column_name]
#                 df[column_name] = df[column_name].clip(lower=min_val, upper=max_val)

#             print(f"✅ Imputed '{column_name}' using context columns: {context_cols}")
#             print(f"    ♦ Remaining missing: {df[column_name].isnull().sum()}")

#         except Exception as e:
#             print(f"❌ Error imputing '{column_name}': {e}")

#     return df

```

```

df_imputed_knn_context = impute_with_knn_context(df_imputed_knn_context, correlation_dict, constraints, n_neighbors=5)

```

```

KNN Context Imputation:   3%|          | 1/37 [00:00<00:15,  2.35it/s]
'age' Imputation Summary (Context):
- Imputed values: 206

```

```

- Skipped (no predictors): 0
- Errors: 0
KNN Context Imputation: 5%|██████| 2/37 [00:00<00:14, 2.49it/s]
'latitude' Imputation Summary (Context):
- Imputed values: 105
- Skipped (no predictors): 0
- Errors: 0
KNN Context Imputation: 8%|██████| 3/37 [00:01<00:13, 2.61it/s]
'longitude' Imputation Summary (Context):
- Imputed values: 105
- Skipped (no predictors): 0
- Errors: 0
KNN Context Imputation: 11%|██████| 4/37 [00:01<00:12, 2.67it/s]
'feels_like' Imputation Summary (Context):
- Imputed values: 858
- Skipped (no predictors): 0
- Errors: 0
KNN Context Imputation: 14%|██████| 5/37 [00:01<00:11, 2.70it/s]
'temp_min' Imputation Summary (Context):
- Imputed values: 483
- Skipped (no predictors): 0
- Errors: 0
KNN Context Imputation: 16%|██████| 6/37 [00:02<00:11, 2.74it/s]
'temp_max' Imputation Summary (Context):
- Imputed values: 478
- Skipped (no predictors): 0
- Errors: 0
KNN Context Imputation: 19%|██████| 7/37 [00:02<00:10, 2.81it/s]
'air_pressure' Imputation Summary (Context):
- Imputed values: 492
- Skipped (no predictors): 0
- Errors: 0
KNN Context Imputation: 22%|██████| 8/37 [00:02<00:10, 2.74it/s]
'air_humidity' Imputation Summary (Context):
- Imputed values: 858
- Skipped (no predictors): 0
- Errors: 0
KNN Context Imputation: 24%|██████| 9/37 [00:03<00:11, 2.51it/s]
'wind_speed' Imputation Summary (Context):
- Imputed values: 488
- Skipped (no predictors): 0
- Errors: 0
KNN Context Imputation: 27%|██████| 10/37 [00:03<00:11, 2.29it/s]
'wind_deg' Imputation Summary (Context):
- Imputed values: 858
- Skipped (no predictors): 0
- Errors: 0
KNN Context Imputation: 30%|██████| 11/37 [00:04<00:11, 2.24it/s]
'clouds_sky' Imputation Summary (Context):
- Imputed values: 858
- Skipped (no predictors): 0
- Errors: 0
KNN Context Imputation: 32%|██████| 12/37 [00:04<00:11, 2.18it/s]
'time_encoded' Imputation Summary (Context):
- Imputed values: 105
- Skipped (no predictors): 0
- Errors: 0

```

```
missing_data_report(df_imputed_knn_context)
```

[Show hidden output](#)

```

def impute_with_knn_no_context(df, constraints={}, n_neighbors=5):
    """
    KNN imputation using all other columns as predictors.
    Optimized: fits KNN once per target column and returns the modified df.
    """
    for column_name in tqdm(df.columns, desc="KNN No-Context Imputation"):
        missing_mask = df[column_name].isna()
        imputed_count = 0
        skipped_count = 0
        errors = []

        if missing_mask.sum() == 0:
            continue

        # Use all other columns as predictors
        predictors = [col for col in df.columns if col != column_name]
        if not predictors:
            skipped_count += 1
            continue

        # Subset dataframe for KNN
        df_subset = df[predictors + [column_name]]

        try:
            imputer = KNNImputer(n_neighbors=n_neighbors)
            imputed_array = imputer.fit_transform(df_subset)
            imputed_df = pd.DataFrame(imputed_array, columns=df_subset.columns, index=df_subset.index)

            # Update only missing values
            mask = df[column_name].isna()
            df.loc[mask, column_name] = imputed_df.loc[mask, column_name]
            imputed_count = mask.sum() - df[column_name].isna().sum()

            # Apply constraints
            if column_name in constraints:
                lower, upper = constraints[column_name]
                df[column_name] = df[column_name].clip(lower=lower, upper=upper)

        except Exception as e:
            errors.append(str(e))

        print(f"\n'{column_name}' Imputation Summary (No Context):")
        print(f" - Imputed values: {imputed_count}")
        print(f" - Skipped (no predictors): {skipped_count}")
        print(f" - Errors: {len(errors)}")
        if errors:
            for err in errors[:5]:
                print(f" - Error: {err}")

    return df

```

```

# def impute_with_knn_no_context(df, constraints=None, n_neighbors=5):
#     # Copy to avoid modifying original
#     df_result = df.copy()

#     try:

```



```
# imputer = KNNImputer(n_neighbors=n_neighbors)
# imputed_array = imputer.fit_transform(df_result)

# # Put back imputed values
# df_imputed = pd.DataFrame(imputed_array, columns=df_result.columns, index=df_result.index)

# # Apply constraints if provided
# if constraints:
#     for col, (min_val, max_val) in constraints.items():
#         if col in df_imputed.columns:
#             df_imputed[col] = df_imputed[col].clip(lower=min_val, upper=max_val)

# print("✅ Imputation without context done.")
# return df_imputed

# except Exception as e:
#     print(f"❌ Error during KNN imputation without context: {e}")
#     return df_result # return original in case of failure
```

```
df_imputed_knn_no_context = impute_with_knn_no_context(df_imputed_knn_no_context, constraints=constraints, n_neighbors=5)
```

```
KNN No-Context Imputation: 3%|█| | 1/37 [00:00<00:14, 2.47it/s]
'age' Imputation Summary (No Context):
- Imputed values: 206
- Skipped (no predictors): 0
- Errors: 0
KNN No-Context Imputation: 5%|█| | 2/37 [00:00<00:13, 2.61it/s]
'latitude' Imputation Summary (No Context):
- Imputed values: 105
- Skipped (no predictors): 0
- Errors: 0
KNN No-Context Imputation: 8%|██| | 3/37 [00:01<00:12, 2.63it/s]
'longitude' Imputation Summary (No Context):
- Imputed values: 105
- Skipped (no predictors): 0
- Errors: 0
KNN No-Context Imputation: 11%|███| | 4/37 [00:01<00:12, 2.67it/s]
'feels_like' Imputation Summary (No Context):
- Imputed values: 858
- Skipped (no predictors): 0
- Errors: 0
KNN No-Context Imputation: 14%|████| | 5/37 [00:01<00:11, 2.73it/s]
'temp_min' Imputation Summary (No Context):
- Imputed values: 483
- Skipped (no predictors): 0
- Errors: 0
KNN No-Context Imputation: 16%|█████| | 6/37 [00:02<00:11, 2.73it/s]
'temp_max' Imputation Summary (No Context):
- Imputed values: 478
- Skipped (no predictors): 0
- Errors: 0
KNN No-Context Imputation: 19%|██████| | 7/37 [00:02<00:10, 2.79it/s]
'air_pressure' Imputation Summary (No Context):
- Imputed values: 492
- Skipped (no predictors): 0
- Errors: 0
KNN No-Context Imputation: 22%|███████| | 8/37 [00:02<00:10, 2.84it/s]
'air_humidity' Imputation Summary (No Context):
- Imputed values: 858
```

```

- Skipped (no predictors): 0
- Errors: 0
KNN No-Context Imputation: 24%|██████████| 9/37 [00:03<00:09, 2.89it/s]
'wind_speed' Imputation Summary (No Context):
- Imputed values: 488
- Skipped (no predictors): 0
- Errors: 0
KNN No-Context Imputation: 27%|██████████| 10/37 [00:03<00:09, 2.97it/s]
'wind_deg' Imputation Summary (No Context):
- Imputed values: 858
- Skipped (no predictors): 0
- Errors: 0
KNN No-Context Imputation: 30%|██████████| 11/37 [00:03<00:08, 3.00it/s]
'clouds_sky' Imputation Summary (No Context):
- Imputed values: 858
- Skipped (no predictors): 0
- Errors: 0
KNN No-Context Imputation: 32%|██████████| 12/37 [00:04<00:08, 3.02it/s]
'time_encoded' Imputation Summary (No Context):
- Imputed values: 195

```

```
missing_data_report(df_imputed_knn_no_context)
```

```

Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []

```

```

Group 2: Columns with >40% to ≤70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []

```

```

Group 3: Columns with >20% to ≤40% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []

```

```

Group 4: Columns with ≤20% missing data
      Column  Missing %  Data Type
      age           0.0    float64
      latitude       0.0    float64
      longitude       0.0    float64
      feels_like      0.0    float64
      temp_min        0.0    float64
      temp_max        0.0    float64
      air_pressure    0.0    float64
      air_humidity    0.0    float64
      wind_speed      0.0    float64
      wind_deg        0.0    float64
      clouds_sky      0.0    float64
      time_encoded    0.0    float64
      suicide_day      0.0    float64
      suicide_month    0.0    float64
      suicide_year     0.0    float64
      suicide_week     0.0    float64
      age_group_encoded 0.0    float64
      financial_condition_encoded 0.0    float64
      weather_main_encoded 0.0    float64
      suicide_weekday_encoded 0.0    float64
      profession_group_encoded 0.0    float64
      hometown_encoded 0.0    float64

```

reason_encoded	0.0	float64
method_encoded	0.0	float64
weather_description_encoded	0.0	float64
marital_status_encoded	0.0	float64
gender_encoded_female	0.0	float64
gender_encoded_male	0.0	float64
gender_encoded_third gender	0.0	float64
workplace_encoded_rural	0.0	float64
workplace_encoded_semi-urban	0.0	float64
workplace_encoded_urban	0.0	float64
religion_encoded_buddhist	0.0	float64
religion_encoded_christian	0.0	float64
religion_encoded_hindu	0.0	float64
religion_encoded_indigenous	0.0	float64
religion_encoded_muslim	0.0	float64

✓ After imputation processing

```
import numpy as np

# Define one-hot groups
one_hot_groups = {
    'gender': [
        'gender_encoded_female',
        'gender_encoded_male',
        'gender_encoded_third gender'
    ],
    'workplace': [
        'workplace_encoded_rural',
        'workplace_encoded_semi-urban',
        'workplace_encoded_urban'
    ],
    'religion': [
        'religion_encoded_buddhist',
        'religion_encoded_christian',
        'religion_encoded_hindu',
        'religion_encoded_indigenous',
        'religion_encoded_muslim'
    ]
}

# Function to fix one-hot encoding
def fix_one_hot_group(df, columns):
    data = df[columns].to_numpy()
    max_indices = np.argmax(data, axis=1) # index of max value in each row

    # Set the max column to 1, others to 0
    fixed_data = np.zeros_like(data)
    fixed_data[np.arange(data.shape[0]), max_indices] = 1

    df[columns] = fixed_data
    return df
```

✖ fixing one hot columns for context data

```
# Apply fix to each group
for group_cols in one_hot_groups.values():
    df_imputed_knn_context = fix_one_hot_group(df_imputed_knn_context, group_cols)
```

✖ fixing one hot columns for no context data

```
# Apply fix to each group
for group_cols in one_hot_groups.values():
    df_imputed_knn_no_context = fix_one_hot_group(df_imputed_knn_no_context, group_cols)
```

```
# Define your constraints
ordinal_constraints = {
    'suicide_weekday_encoded': (0, 6),
    'age_group_encoded': (0, 6),
    'financial_condition_encoded': (0, 4),
    'weather_main_encoded': (0, 7),
}
```

✖ fixing ordinal columns for context data

```
# Apply rounding and clip to valid range
for col, (low, high) in ordinal_constraints.items():
    if col in df_imputed_knn_context.columns:
        df_imputed_knn_context[col] = df_imputed_knn_context[col].round().clip(lower=low, upper=high)
```

✖ fixing ordinal columns for no context data

```
# Apply rounding and clip to valid range
for col, (low, high) in ordinal_constraints.items():
    if col in df_imputed_knn_context.columns:
        df_imputed_knn_no_context[col] = df_imputed_knn_no_context[col].round().clip(lower=low, upper=high)
```

```
df_imputed_knn_context.tail(5)
```

	age	latitude	longitude	feels_like	temp_min	temp_max	air_pressure	air_humidity	wind_speed	wind_deg	...	gender_encoded_male	gender_encoded_third gender	workplace_enc
1612	15.0	23.894376	90.166603	300.650	299.922	300.216	1004.6	82.0	3.000	138.6	...	0.0	0.0	
1613	27.0	24.309447	88.987565	300.650	299.288	299.288	1003.4	86.2	3.000	135.8	...	0.0	0.0	
1614	9.0	23.409802	89.384499	300.650	299.944	299.944	1002.4	86.6	3.726	142.0	...	1.0	0.0	
1615	24.0	23.898795	89.129554	291.776	12.500	23.700	1019.0	77.4	8.000	94.2	...	0.0	0.0	
1616	26.0	22.655478	89.794181	291.544	29.800	36.600	1003.8	78.6	3.800	84.6	...	0.0	0.0	

5 rows × 37 columns

df_imputed_knn_no_context.tail(5)

	age	latitude	longitude	feels_like	temp_min	temp_max	air_pressure	air_humidity	wind_speed	wind_deg	...	gender_encoded_male	gender_encoded_third gender	workplace_enc
1612	15.0	23.894376	90.166603	300.650	299.922	300.216	1004.6	82.0	3.000	138.6	...	0.0	0.0	
1613	27.0	24.309447	88.987565	300.650	299.288	299.288	1003.4	86.2	3.000	135.8	...	0.0	0.0	
1614	9.0	23.409802	89.384499	300.650	299.944	299.944	1002.4	86.6	3.726	142.0	...	1.0	0.0	
1615	24.0	23.898795	89.129554	291.776	12.500	23.700	1019.0	77.4	8.000	94.2	...	0.0	0.0	
1616	26.0	22.655478	89.794181	291.544	29.800	36.600	1003.8	78.6	3.800	84.6	...	0.0	0.0	

5 rows × 37 columns

```
# # prompt: give me the code make the latest df downloadable and download that db named final_suicide_cleaned.xlsx ins excel format

# # Make the latest DataFrame downloadable
# from google.colab import files

# # Specify the filename for the Excel file
# filename1 = '8_knn_context.xlsx'

# # Save the DataFrame to an Excel file
# df_imputed_knn_context.to_excel(filename1, index=False)

# # Download the file
# files.download(filename1)
```

```
# # prompt: give me the code make the latest df downloadable and download that db named final_suicide_cleaned.xlsx ins excel format

# # Make the latest DataFrame downloadable
# from google.colab import files

# # Specify the filename for the Excel file
# filename2 = '9_knn_no_context.xlsx'
```

```
# # Save the DataFrame to an Excel file
# df_imputed_knn_no_context.to_excel(filename2, index=False)

# # Download the file
# files.download(filename2)
```